Space Shuttle Main Engine

November 11, 2026

Contents

1 Introduction **1**

2 Oxygen **2**
2.1 First Enthalpy Exchanger 2
2.2 Combustion Chamber 4
2.3 Nozzle Performance 9

A Code listings **10**
A.1 SSMEMain.py 10
A.2 exchanger.py 11
A.3 Iterater.py 19

1 Introduction

The Space Shuttle Main Engine (SSME) is a liquid-fueled rocket engine used to power the Space Shuttle Orbiter. This report will discuss the design and methodologies of the SSME code, as well as the results of the code. For this project, we were given the parameters below:

Combustion Chamber Data	
Fuel	Hydrogen
Oxidizer	O_2
Fuel/Oxidizer Ratio	0.166
Hydrogen Injection Temperature	850 K
Oxidizer Injection Temperature	530 K
Chamber Pressure	204 atm

R_t	5.15 inch
R_e	45.35 inch
θ_p	32 degree

2 Oxygen

2.1 First Enthalpy Exchanger

We begin with computing the equivalence ratio ϕ for the oxygen first enthalpy exchanger. The equivalence ratio is defined as the ratio of the mass of fuel to the mass of oxidizer. The mass of fuel is the mass of hydrogen, and the mass of oxidizer is the mass of oxygen. The mass of hydrogen is $\phi \frac{mol_{O_2}}{mol_{H_2}} = \Phi$.

From here we are to calculate the stndard heat of formation by:

$$\Delta\hat{h}_{H_2} = \hat{h}_{H_2}(850) + \hat{R}T^2 \int_{pf}^{pi} \frac{1}{p} \frac{-\hat{b}_{H_2} p}{\hat{R}T^2} dp \tag{1}$$

$$\Delta\hat{h}_{O_2} = \hat{h}_{O_2}(530) + \hat{R}T^2 \int_{pf}^{pi} \frac{1}{p} \frac{-\hat{b}_{O_2} p}{\hat{R}T^2} dp \tag{2}$$

Species	$\hat{b}$
H_2	0.0266
O_2	0.0318
H_2O	0.03049
N_2	0.0391
F_2	0.02896
HF	0.0739

Where $\hat{b}$ is read from a table developed in `exchanger.py` file. $\Delta\hat{h}_i$ is calculated from both hydrogen and oxygen. Once we calculate $\Delta\hat{h}_i$ for each, we can calculate the enthalpy of the mixture.

$$\Delta\hat{h}_{mix} = \Phi\Delta\hat{h}_{H_2} + \Delta\hat{h}_{O_2} \tag{3}$$

We can then apply this to our chemical equation.

$$O_2 + \Phi H_2 \rightarrow (2-\Phi)H_2 + 2H_2O \tag{4}$$

Note: notice the $(2-\Phi)$ in the equation above. This is because we calculated the enthalpy of formation for hydrogen and not hydrogen gas H_2.

From here we can sum the molar enthalpies of formation to get the first estimate of the first reaction enthalpy

$$\Delta\hat{h}_r = \Phi\Delta\hat{h}_{H_2} + 2(\Delta\hat{h}_{H_2O}) \tag{5}$$

From here we go into our Temperature finding Algorithm. We are given the following parameters:

$$r = \Phi - 1 \tag{6}$$

$$q = 1 \tag{7}$$

$$r_{percent} = \frac{r}{r+q} \tag{8}$$

$$q_{percent} = \frac{q}{r+q} \tag{9}$$

From here we use the values of (Eq. 6) - (Eq. 9) to calculate the temperature of the first enthalpy exchanger.

$$T_{H_2} = \texttt{h2_tableT}(\Delta\hat{h}_r) \tag{10}$$

$$T_{H_2O} = \texttt{ho2_tableT}(q_{percent}\Delta\hat{h}_r) \tag{11}$$

Once we have returned the values of Temperature, we then average the two values to get the final temperature of the first enthalpy exchanger.

$$T_C = \frac{T_{H_2} + T_{H_2O}}{2} \tag{12}$$

2.2 Combustion Chamber

For the Combustion Chamber, we have the two equations below:

$$H_2O \rightarrow OH + H \tag{13}$$

$$H_2 \rightarrow 2H \tag{14}$$

The Gibbs Free Energy for each species is given by:

$$\hat{g}_{H_2O} = \texttt{h2o_table_g}(T_C) \tag{15}$$

$$\hat{g}_{H_2} = \texttt{h2_table_g}(T_C) \tag{16}$$

$$\hat{g}_{OH} = \texttt{oh_table_g}(T_C) \tag{17}$$

$$\hat{g}_{H} = \texttt{h_table_g}(T_C) \tag{18}$$

We can then calculate the Gibbs Free Energy of the reaction by:

$$\Delta\hat{g}_r = 2(\hat{g}_H - \hat{g}_{H_2}) \tag{19}$$

$$\Delta\hat{g}_r = \hat{g}_H - \hat{g}_{OH} + \hat{g}_{H_2O} \tag{20}$$

Now is the point where we involve the **Law of Mass Action**:

Species	$\nu'_{(i)}$	$\nu''_{(i)}$	$\nu''_{(i)} - \nu''_{(i)}$
H_2O	1	0	-1
OH	0	1	1
H	0	1	1

We an now use:

$$K_n = \prod_{i=1}^{N_s} X_{(n)}^{(\nu''_{(i)} - \nu'_{(i)})} = (\frac{p}{p_a})^{-\sigma_v} K_p \tag{21}$$

Where;

$$\sigma_v = \sum_{i=1}^{N_s} (\nu''_{(i)} - \nu'_{(i)}) \tag{22}$$

$$K_p = e^{-\frac{\Delta G}{\hat{R}T}} \tag{23}$$

$$\Delta G = \sum_{i=1}^{N_s} (\nu''_{(i)} \hat{g}_{(i)}) \hat{g}_{(i)} \tag{24}$$

From here we can solve for the equilibrium constant K_n.

$$K_{H_2O} = (\frac{204atm}{1atm})^{-\frac{\Delta G_{H_2O}}{\hat{R}T_C}} \tag{25}$$

$$K_{H_2} = (\frac{204atm}{1atm})^{-\frac{\Delta G_{H_2}}{\hat{R}T_C}} \tag{26}$$

Now we move into the chemistry problem.
Atom Balance:

$$H_2 + lO_2 \rightarrow N_{H_2O} H_2O + N_{H_2} H_2 + N_H H + N_{OH} OH \tag{27}$$

Recall, due to the fule being Hydrogen;

$$mass_{fuel} = 2.02 \tag{28}$$

$$mass_{Oxidizer} = \frac{mass_{fuel}}{\phi} \tag{29}$$

$$l = \frac{mass_{Oxidizer}}{2\hat{m}_{O_2}} \tag{30}$$

Where $\hat{m}_{O_2}$ is the molar mass of Oxygen. And ϕ is the Fuel to Oxidizer ratio, and not the equivalence ratio. Revisiting the **Atom Balance** equation, we can now solve for the number of moles of each species.

For $H_2O \rightarrow OH + H$:

$$H_2 : 2 = 2N_{H_2O} + 2N_{H_2} + N_H + N_{OH} \tag{31}$$

$$O_2 : 2l = N_{H_2O} + N_{OH} \tag{32}$$

$$1 = N_{H_2O} + N_{H_2} + \frac{1}{2}N_H + \frac{1}{2}N_{OH} \tag{33}$$

Where $\frac{1}{2}N_H + \frac{1}{2}N_{OH} \rightarrow 0$. Resulting in:

$$1 = N_{H_2O} + N_{H_2} \tag{34}$$

$$2(0.38050899) = N_{H_2O} + N_{OH} \tag{35}$$

Where $N_{OH} \rightarrow 0$. Resulting in:

$$0.760592 = N_{H_2O} \tag{36}$$

$$X = \frac{N_{Molecule}}{N_{Total}} \tag{37}$$

$$X_{H_2O} = \frac{N_{H_2O}}{N_{Total}} \tag{38}$$

$$X_H = \frac{N_H}{N_{Total}} \tag{39}$$

$$X_{OH} = \frac{N_{OH}}{N_{Total}} \tag{40}$$

$$X_{H_2O} = \frac{N_{H_2O}}{N_{H_2O} + N_{H_2} + N_H + N_{OH}} \tag{41}$$

$$X_H = \frac{N_H}{N_{H_2O} + N_{H_2} + N_H + N_{OH}} \tag{42}$$

$$X_{OH} = \frac{N_{OH}}{N_{H_2O} + N_{H_2} + N_H + N_{OH}} \tag{43}$$

Recall, $N_{OH} \rightarrow 0$ and $N_H \rightarrow 0$. Resulting in:

$$K_{H_2O} = \frac{\frac{N_{OH}}{N_{H_2O}+N_{H_2}} \frac{N_H}{N_{H_2O}+N_{H_2}}}{\frac{N_{H_2O}}{N_{H_2O}+N_{H_2}}} \tag{44}$$

Resulting in,

$$K_{H_2O} = \frac{N_H N_{OH}}{(N_H + N_{H_2O})N_{H_2O}} \tag{45}$$

Similarly we follow the same steps for the $H_2 \rightarrow H + H$ reaction.

$$X_{H_2} = \frac{N_{H_2}}{Total} \tag{46}$$

$$X_H = \frac{N_H}{Total} \tag{47}$$

$$X_{H_2} = \frac{N_{H_2}}{N_{H_2} + N_{H_2O} + N_H + N_{OH}} \tag{48}$$

$$X_H = \frac{N_H}{N_{H_2} + N_{H_2O} + N_H + N_{OH}} \tag{49}$$

$$K_{H_2} = \frac{(\frac{N_H}{N_{H_2O}+N_{H_2}})^2}{\frac{N_{H_2}}{N_{H_2O}+N_{H_2}}} \tag{50}$$

$$K_{H_2} = \frac{N_H^2}{(N_{H_2O} + N_{H_2})N_{H_2}} \tag{51}$$

As a result we have 4 coupled equations:

$$1 = N_{H_2O} + N_{H_2} \tag{52}$$

$$0.760592 = N_{H_2O} \tag{53}$$

$$K_{H_2O} = \frac{N_H N_{OH}}{(N_H + N_{H_2O})N_{H_2O}} \tag{54}$$

$$K_{H_2} = \frac{N_H^2}{(N_{H_2O} + N_{H_2})N_{H_2}} \tag{55}$$

Where K_{H_2O} and K_{H_2} are calculated values.
Solving these equations we get:

$$N_{H_2O} = 0.760592 \tag{56}$$

$$N_{H_2} = 0.239408 \tag{57}$$

$$N_H = 0.05333 \tag{58}$$

$$N_{OH} = 0.101958 \tag{59}$$

Species	$N_{(i)}$	$X_{(i)}$	$m_{(i)}$	$Y_{(i)}$	$\hat{c}_{p(i)}$	$c_{p(i)}$	$Y_{(i)}c_p$
H_2O	N_{H_2O}	$X_{H_2O} = \frac{N_{H_2O}}{N_{Total}}$	$\hat{m}N_{H_2O}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H_2O)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{H_2O}c_p$
H_2	N_{H_2}	$X_{H_2} = \frac{N_{H_2}}{N_{Total}}$	$\hat{m}N_{H_2}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H_2)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{H_2}c_p$
H	N_H	$X_H = \frac{N_H}{N_{Total}}$	$\hat{m}N_H$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	Y_Hc_p
OH	N_{OH}	$X_{OH} = \frac{N_{OH}}{N_{Total}}$	$\hat{m}N_{OH}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(OH)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{OH}c_p$

Now we can calculate c_p, we sum the $Y_{(i)}c_p$ values for each species.

$$c_p = \sum_i Y_{(i)}c_p \tag{60}$$

$$c_p = Y_{H_2O}c_p + Y_{H_2}c_p + Y_Hc_p + Y_{OH}c_p \tag{61}$$

We perform now calulate the gas constant R.

$$R = \frac{\hat{R}}{m_{Total}} \tag{62}$$

where $m_{Total} = \sum_i X_{(i)}m_{(i)}$ is the mixture molar mass. The mole fractions are normalized by $N_{Total} = N_{H_2O} + N_{H_2} + N_H + N_{OH}$, which exceeds one once dissociation produces H and OH.

$$c_v = c_p - R \tag{63}$$

$$\gamma = \frac{c_p}{c_p - \frac{\hat{R}}{\sum_i X_{(i)}m_{(i)}}} = \frac{c_p}{c_p - \frac{\hat{R}}{m_{Total}}} = \frac{c_p}{c_p - R} = \frac{c_p}{c_v} \tag{64}$$

2.3 Nozzle Performance

We begin by defining the Area Ratio:

$$A_r = \frac{\pi R_e^2}{\pi R_t^2} \tag{65}$$

Allowing for us to solve for the Mach number:

$$A_r = \left(\frac{\gamma+1}{2}\right)^{-\frac{\gamma+1}{2(\gamma-1)}} \frac{1}{M_e}\left[1+\frac{\gamma-1}{2}M_e^2\right]^{\frac{\gamma+1}{2(\gamma-1)}} \tag{66}$$

We can solve for Mach number M_e by making use of the `scipy.optimize.fsolve` function, which returns the roots of a non-linear function. After solving for M_e we can calculate the exit velocity v_e, exit Pressure p_e and exit Temperature T_e.

$$T_e = T_c\left[1+\frac{\gamma-1}{2}M_e^2\right]^{-1} \tag{67}$$

$$p_e = p_c\left[1+\frac{\gamma-1}{2}M_e^2\right]^{-\frac{\gamma}{\gamma-1}} \tag{68}$$

$$v_e = M_e\sqrt{\gamma R T_e} \tag{69}$$

After solving for (Eq. 69), (Eq. 68) and (Eq. 67) we can calculate the mass flow rate $\dot{m}$.

$$\dot{m} = \frac{A_t p_c \sqrt{\gamma}}{\sqrt{RT_c}}\left[\frac{2}{\gamma+1}\right]^{\frac{\gamma+1}{2(\gamma-1)}} \tag{70}$$

Then once you have the mass flow rate you can calculate the thrust $\mathcal{T}$, specific impulse I_{sp} and the thrust coefficient C_T.

$$\mathcal{T} = \dot{m}v_e + (p_e - p_a)A_e \tag{71}$$

where p_a is the ambient pressure ($p_a = 0$ in vacuum, $p_a = 101325$ Pa at sea level).

$$I_{sp} = \frac{\mathcal{T}}{\dot{m}g_0} \tag{72}$$

$$C_T = \frac{\mathcal{T}}{p_c A_t} \tag{73}$$

A Code listings

A.1 SSMEMain.py

```
import pandas as pd
from exchanger import *


if __name__ == '__main__':

    p_starting = 204 #starting pressure in atm
    p_starting = p_starting * 101325 #convert to pascals
    FuelOxidizer_ratio = 0.166
    mol_O2 = 31.998 # g/mol #single molecule mass
    mol_H2 = 2.016 # g/mol #single molecule mass
    phi = (FuelOxidizer_ratio* mol_O2 / mol_H2)/2
    Hydr_inject_temp = 850 #K
    Ox_inject_temp = 530   #K
    ThroatRadius = 5.15 #inches
    ExitRadius = 45.35 #inches
    FEE_T, FEE_del_h_total, del_h2_hat, b_hat_table = FEE(Hydr_inject_temp, p_starting, Ox_inject_temp, phi)
    print('phi = ', phi)

    print('Temperature of the first enthalpy exchanger = ', FEE_T, 'K')
    print('Total enthalpy change in the first enthalpy exchanger = ', FEE_del_h_total, 'J')


    gamma, R, cp_total, cv, LawOfMassAction_df = OCC(FEE_T, p_starting, FuelOxidizer_ratio)
    print('gamma = ', gamma)

    p_sea_level = 101325 # Pa
    m_dotOx, v_eOx, p_eOx, ThrustOx, c_TOx, I_spOx, T_e_Ox, M_eOx = OxNozzlePerf(ThroatRadius, ExitRadius, gamma, FEE_T, R, p_starting)
    _, _, _, ThrustOx_SL, c_TOx_SL, I_spOx_SL, _, _ = OxNozzlePerf(ThroatRadius, ExitRadius, gamma, FEE_T, R, p_starting, p_sea_level)

    print('Ox exit Mach number = ', M_eOx)
    print('Ox mass flow rate = ', m_dotOx, 'kg/s')
    print('Ox exit velocity = ', v_eOx, 'm/s')
```

```
    print('Ox exit pressure = ', p_eOx, 'Pa')
    print('Ox exit temperature = ', T_e_Ox, 'K')
    print('Ox thrust (vacuum) = ', ThrustOx, 'N')
    print('Ox thrust coefficient (vacuum) = ', c_TOx)
    print('Ox specific impulse (vacuum) = ', I_spOx, 's')
    print('Ox thrust (sea level) = ', ThrustOx_SL, 'N')
    print('Ox thrust coefficient (sea level) = ', c_TOx_SL)
    print('Ox specific impulse (sea level) = ', I_spOx_SL, 's')


    print('Flourine Problem')
    b_hat_table, FEE_T_flourine = FEE_F(phi, p_starting, del_h2_hat, b_hat_table, Ox_inject_temp)

    print('Temperature of the first enthalpy exchanger (Flourine) = ', FEE_T_flourine, 'K')

    FCC(FEE_T_flourine)
```

A.2 exchanger.py

```
from Iterater import *
import pandas as pd
import numpy as np
from sympy import Eq, solve, var


def FEE(Hydr_inject_temp, p_starting, Ox_inject_temp, phi):
    '''
    This is a function that returns the calculations of the first enthalpy exchanger
    :param Hydr_inject_temp: temperature of hydrogen injection in K
    :param p_starting: starting pressure in Pa
    :param Ox_inject_temp: temperature of oxygen injection in K
    :param phi: equivalence ratio
    :return T the temperature of the first enthalpy exchanger
    :return del_h_total the total enthalpy change in the first enthalpy exchanger
    :return b_hat_table the table of b_hat values
    :return del_h2_hat the enthalpy change of H2
    '''

```

```
20
21      h_f_h2o = abs(h2o_table_f(298)) #J/mol
22      h_f_h = h_table_f(298)/1000
23      h_oh_f = oh_table_f(298)/1000  # KJ/mol
24
25      # Van der Waals co-volume b_hat (L/mol)
26      b_hat_table = pd.DataFrame({'substance': ['H2', 'H2O', '
        O2', 'N2'],
27                                  'b_hat': [0.0266, 0.03049,
        0.0318, 0.0391]})
28
29      b_hat_h2 = b_hat_table['b_hat'][0]
30      b_hat_o2 = b_hat_table['b_hat'][2]
31
32      del_h2_hat = h2_tableH(Hydr_inject_temp) + b_hat_h2*(
        p_starting - 101325)/1000
33      del_o2_hat = o2_tableH(Ox_inject_temp) + b_hat_o2*(
        p_starting - 101325)/1000
34      mols_h2 = phi*2
35      mols_o2 = 1
36
37      del_h_total = del_h2_hat*mols_h2 + del_o2_hat*mols_o2
38      # At this point the chemical equation is O2 + mols*H2 ->
        (phi-2)*H2 +2H2O
39      del_h_total = del_h_total + 2*h_f_h2o
40
41      # weighting function
42      r = 0.17947269916082
43      q = (1- r)/2
44
45      T_h2o = h2o_tableT(q*del_h_total)
46      T_h2 = h2_tableT(r*del_h_total)
47
48      T = (T_h2o + T_h2)/2
49
50      return T, del_h_total, del_h2_hat, b_hat_table
51
52  def OCC(FEE_T, p_starting, FO_ratio):
53      '''
54      This is a function that returns the calculations of the
        oxygen combustion chamber
55      :param FEE_T: temperature of the first enthalpy exchanger
        in K
56      :param p_starting: starting pressure in Pa
57      :param FO_ratio: fuel to oxidizer ratio
```

```
    :return gamma: the ratio of specific heats
    :return R: the gas constant
    :return cp_total: the total specific heat capacity
    :return cv : the specific heat capacity at constant volume
    :return LawOfMassAction_df: the dataframe of the law of mass action
    '''

    # The chemical equation is H2O -> H + OH
    molfrac_h2o_react = 1
    molfrac_h_react = 0
    molfrac_oh_react = 0
    molfrac_h2o_prod = 0
    molfrac_h_prod = 1
    molfrac_oh_prod = 1

    # The chemical equation is H2 -> 2H
    molfrac_h2_react = 1
    molfrac_2h_react = 0
    molfrac_2h_prod = 2
    molfrac_h2_prod = 0

    molfrac_h2o_total = molfrac_h2o_prod - molfrac_h2o_react
    molfrac_h_total = molfrac_h_prod - molfrac_h_react
    molfrac_oh_total = molfrac_oh_prod - molfrac_oh_react
    sigma_nu = molfrac_h2o_total + molfrac_h_total + molfrac_oh_total
    # print('sigma_nu = ', sigma_nu)

    molfrac_h2_total = molfrac_h2_prod - molfrac_h2_react
    molfrac_2h_total = molfrac_2h_prod - molfrac_2h_react
    sigma_nu = molfrac_h2_total + molfrac_2h_total
    # print('sigma_nu = ', sigma_nu)

    # Gibbs free energy of formation
    g_f_h2o = h2o_table_g(FEE_T)
    g_f_h = h_table_g(FEE_T)
    g_f_oh = oh_table_g(FEE_T)
    g_f_h2 = h2_table_g(FEE_T)

    # Gibbs free energy of reaction
    g_r_h2 = 2*(g_f_h - g_f_h2)
    g_r_h2o = g_f_h + g_f_h2o - g_f_oh

```

```
    k_h2 = ((p_starting/101325)**-1)*np.exp(-g_r_h2/(8.314*FEE_T))
    k_h2o = ((p_starting/101325)**-1)*np.exp(-g_r_h2o/(8.314*FEE_T))


    # k_h2o = 0.007149
    # k_h2 = 0.011874839

    mass_fuel = 2.02 #H2
    mass_oxider = mass_fuel/FO_ratio
    l = mass_oxider/ (2*15.99)

    N_h = var('N_h')
    N_oh = var('N_oh')
    N_h2 = var('N_h2')
    N_h2o = var('N_h2o')


    # solve the system of equations - Law of mass action
    sol = solve([Eq(N_h2o, 2*l), Eq(N_h2o+N_h2, 1), Eq((N_h*N_oh)/((N_h2+N_h2o)*N_h2o), k_h2o),
                 Eq((N_h**2)/((N_h2+N_h2o)*N_h2), k_h2)], [N_h, N_oh, N_h2, N_h2o])
    # split sol into two separate lists
    sol1 = sol[0]
    sol2 = sol[1]

    # if all the solutions are real and positive then assign mols_sol to the solution
    if all(i > 0 for i in sol1) and all(i > 0 for i in sol2):
        mols_sol = sol1
    else:
        mols_sol = sol2

    # print('mols_sol = ', mols_sol)
    N_h = mols_sol[0]
    N_oh = mols_sol[1]
    N_h2 = mols_sol[2]
    N_h2o = mols_sol[3]
    N_total = N_h + N_oh + N_h2 + N_h2o

    molar_mass_h2o = 18.01528
    molar_mass_h = 1.00794
    molar_mass_oh = 17.00734
```

```
    molar_mass_h2 = 2.01588

    LawOfMassAction_df = pd.DataFrame({'substance': ['N_H2O',
    'N_H2', 'N_H', 'N_OH', 'Total'],
                                       'mols': [N_h2o, N_h2,
    N_h, N_oh, N_total]})
    # Mole fractions must be normalized by the total moles:
    once H and OH form,
    # N_total = N_H2O + N_H2 + N_H + N_OH > 1, so X_(i) != N_
    (i).
    LawOfMassAction_df['X_(i)'] = LawOfMassAction_df['mols']
    / N_total
    # X_(i)*m_hat_(i): the species' contribution to the
    mixture molar mass (g/mol)
    LawOfMassAction_df['m_hat_(i)'] = pd.Series([
    molar_mass_h2o, molar_mass_h2, molar_mass_h, molar_mass_oh
    , 0.0])\
                                      * LawOfMassAction_df['
    X_(i)']

    # mixture molar mass m_hat = sum_i X_(i) m_hat_(i), over
    the four species
    m_hat_total = sum(LawOfMassAction_df['m_hat_(i)'][0:4])
    LawOfMassAction_df.loc[4, 'm_hat_(i)'] = m_hat_total
    # mass fractions Y_(i) = X_(i) m_hat_(i) / m_hat
    LawOfMassAction_df['Y_(i)'] = LawOfMassAction_df['m_hat_(
    i)'] / m_hat_total

    h2_cp = h2_table_cp(FEE_T)
    h2o_cp = h2o_table_cp(FEE_T)
    h_cp = h_table_cp(FEE_T)
    oh_cp = oh_table_cp(FEE_T)

    cp_hats = pd.Series([h2o_cp, h2_cp, h_cp, oh_cp])
    cps = cp_hats / pd.Series([molar_mass_h2o, molar_mass_h2,
    molar_mass_h, molar_mass_oh])
    cp_df = pd.DataFrame({'cp_hat': cp_hats, 'cp': cps})
    cp_df['Y_(i)*cp_(i)'] = cp_df['cp'] * LawOfMassAction_df[
    'Y_(i)'][0:4]

    cp_total = sum(cp_df['Y_(i)*cp_(i)'])

    LawOfMassAction_df = pd.concat([LawOfMassAction_df, cp_df
    ], axis=1)

```

```
    R = 8.314/m_hat_total
    cv = cp_total - R

    gamma = cp_total/cv

    return gamma, R, cp_total, cv, LawOfMassAction_df

def FEE_F(phi, p_starting, del_h2_hat, b_hat_table,
    Ox_inject_temp):
    '''
    This is a function that returns the calculations of the
    fuel enthalpy exchanger - flourine
    :param phi the equivalence ratio
    :param p_starting combustion pressure in Pa
    :param del_h2_hat heat of formation of H2 from FEE -
    Oxygen
    :param b_hat_table Van der Waals table of constants
    :param Ox_inject_temp: Oxidizer injection temperature in
    K
    :return T the temperature of the FEE in K
    :return b_hat_table the Van der Waals table of constants
    - edited
    '''

    b_hat_table = pd.concat([b_hat_table,
                             pd.DataFrame({'substance': ['F2'
    , 'HF'], 'b_hat': [0.02896, 0.0739]})],
                            ignore_index=True)

    b_hat_f2 = b_hat_table['b_hat'][3]
    F2_inject_temp = Ox_inject_temp

    del_f2_hat = f2_tableH(F2_inject_temp) + b_hat_f2*(
    p_starting - 101325)/1000
    del_hf_hat = abs(hf_tableH(298))
    del_h_total = phi*del_h2_hat + del_f2_hat
    del_h_total = del_h_total + del_hf_hat


    r = phi - 1
    q = 1

    r_perc = r/(r+q)
    q_perc = 1-r_perc

```

```
    T_h2 = h2_tableT(del_h_total)
    T_hf = hf_tableT(q_perc*del_h_total)

    T = (T_hf + T_h2) / 2


    return b_hat_table, T

def OxNozzlePerf(ThroatRadius, ExitRadius, gamma, FEE_T, R,
    p_starting, p_ambient=0.0):
    '''
    This is a function that returns the performance of the
    oxidizer nozzle
    :param ThroatRadius: throat radius in inches
    :param ExitRadius: exit radius in inches
    :param gamma: ratio of specific heats (frozen, from OCC)
    :param FEE_T: chamber temperature in K
    :param R: gas constant in kJ/kg-K
    :param p_starting: chamber pressure in Pa
    :param p_ambient: ambient pressure in Pa (0 = vacuum,
    101325 = sea level)
    :return: m_dot, v_e, p_e, Thrust, c_T, I_sp, T_e, M_e
    '''

    Area_ratio = (ExitRadius/ThroatRadius)**2
    # change np.float64 values to a float
    gamma = float(gamma)
    FEE_T = float(FEE_T)
    R = float(R)
    R = R*1000 # convert to J/kg-K

    M_e = float(MachSolve(gamma, Area_ratio, 5.0)[0])
    T_e = FEE_T*((1+((gamma-1)/2)*M_e**2)**(-1))
    v_e = M_e * np.sqrt(gamma * R * T_e)
    p_e = p_starting * (1 + ((gamma - 1) / 2) * M_e ** 2) **
    (-gamma / (gamma - 1))
    ExitRadius = ExitRadius/39.37 # convert to meters
    ThroatRadius = ThroatRadius/39.37 # convert to


    m_dot = (p_starting * np.pi * (ThroatRadius)**2) / (np.
    sqrt(FEE_T)) * \
        np.sqrt(gamma/R)*((2/(gamma+1))**((gamma+1)/(2*(gamma
    -1))))

```

```
    A_e = np.pi * ExitRadius**2
    A_t = np.pi * ThroatRadius**2

    # Control-volume momentum balance: F = m_dot*v_e + (p_e - p_a)*A_e.
    # Chamber pressure only enters through c_T's normalization (p_c*A_t), not the thrust itself.
    Thrust = m_dot * v_e + (p_e - p_ambient) * A_e
    c_T = Thrust / (p_starting * A_t)
    I_sp = Thrust / (m_dot * 9.81)

    return m_dot, v_e, p_e, Thrust, c_T, I_sp, T_e, M_e

def FCC(FEE_F):
    '''
    This is a function that returns the performance of the flourine combustion chamber
    :return:
    '''


    # The chemical equation is H2 +F2-> 2HF
    molfrac_h2_react = 1
    molfrac_f2_react = 1
    molfrac_hf_react = 0
    molfrac_h2_prod = 0
    molfrac_f2_prod = 0
    molfrac_hf_prod = 2


    molfrac_h2_total = molfrac_h2_react - molfrac_h2_prod
    molfrac_f2_total = molfrac_f2_react - molfrac_f2_prod
    molfrac_hf_total = molfrac_hf_react - molfrac_hf_prod
    sigma_nu = molfrac_hf_total + molfrac_h2_total + molfrac_f2_total
    # print('sigma_nu = ', sigma_nu)

    # Gibbs free energy of formation of H2, F2, and HF
    del_h_h2 = h2_tableH(FEE_F)
    del_h_f2 = f2_tableH(FEE_F)
    del_h_hf = hf_tableH(FEE_F)

    s_h2 = h2_tableS(FEE_F)
    s_f2 = f2_tableS(FEE_F)
    s_hf = hf_tableS(FEE_F)
```

```
289
290     # Gibbs
291     del_g_h2 = del_h_h2 - FEE_F*s_h2
292     del_g_f2 = del_h_f2 - FEE_F*s_f2
293     del_g_hf = del_h_hf - FEE_F*s_hf
294
295     del_g_total = -del_g_f2 - del_g_h2 + del_g_hf
296     print('del_g_total = ', del_g_total)
```

A.3 Iterater.py

```
1  from pathlib import Path
2
3  import numpy as np
4  import pandas as pd
5  from scipy.optimize import fsolve
6
7  # Thermo tables live next to this file, so the code runs from
       any working directory
8  TABLES = Path(__file__).resolve().parent / 'tables'
9
10
11 def h2_tableH(Temp):
12     '''
13     :param Temp: temperature in K
14     :return h_hat: enthalpy in J/mol also known as heat of
       formation
15     '''
16     h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')
17
18     idx = (h2_table['T']-Temp).abs().idxmin()
19     T_cloest = h2_table['T'][idx]
20     h_hat_close = h2_table['H*-H*_298'][idx]
21     if Temp > T_cloest:
22         T_upper = h2_table['T'][idx+1]
23         h_hat_upper = h2_table['H*-H*_298'][idx+1]
24         h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(
       T_upper-T_cloest)*(Temp-T_cloest)
25     else:
26         T_lower = h2_table['T'][idx-1]
27         h_hat_lower = h2_table['H*-H*_298'][idx-1]
28         h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
       T_lower-T_cloest)*(Temp-T_cloest)
29     h_hat = h_hat*4184
30
```

```
    return h_hat

def f2_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    f2_table = pd.read_csv(TABLES / 'Fluorine_F2_table.csv')

    idx = (f2_table['T']-Temp).abs().idxmin()
    T_closest = f2_table['T'][idx]
    h_hat_close = f2_table['H*-H*_298'][idx]
    if Temp > T_closest:
        T_upper = f2_table['T'][idx+1]
        h_hat_upper = f2_table['H*-H*_298'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = f2_table['T'][idx-1]
        h_hat_lower = f2_table['H*-H*_298'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_closest)*(Temp-T_closest)
    h_hat = h_hat*4184

    return h_hat

def o2_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    o2_table = pd.read_csv(TABLES / 'o2_table.csv')

    idx = (o2_table['T']-Temp).abs().idxmin()
    T_closest = o2_table['T'][idx]
    h_hat_close = o2_table['H*-H*_298'][idx]
    if Temp > T_closest:
        T_upper = o2_table['T'][idx+1]
        h_hat_upper = o2_table['H*-H*_298'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = o2_table['T'][idx-1]
```

```
71          h_hat_lower = o2_table['H*-H*_298'][idx-1]
72          h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
73      h_hat = h_hat
74
75      return h_hat
76
77  def hf_tableH(Temp):
78      '''
79      :param Temp: temperature in K
80      :return h_hat: enthalpy in J/mol also known as heat of
    formation
81      '''
82      hf_table = pd.read_csv(TABLES / '
    HydrogenFluoride_HF_table.csv')
83
84      idx = (hf_table['T']-Temp).abs().idxmin()
85      T_cloest = hf_table['T'][idx]
86      h_hat_close = hf_table['h_hat'][idx]
87      if Temp > T_cloest:
88          T_upper = hf_table['T'][idx+1]
89          h_hat_upper = hf_table['h_hat'][idx+1]
90          h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
91      else:
92          T_lower = hf_table['T'][idx-1]
93          h_hat_lower = hf_table['h_hat'][idx-1]
94          h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
95      h_hat = h_hat*4184
96
97      return h_hat
98
99  def h2o_table_f(Temp):
100     '''
101     :param Temp: temperature in K
102     :return h_hat: enthalpy in J/mol also known as heat of
    formation
103     '''
104     h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')
105
106     idx = (h2o_table['T']-Temp).abs().idxmin()
107     T_cloest = h2o_table['T'][idx]
108     h_hat_close = h2o_table['h_hat'][idx]
109     if Temp > T_cloest:
```

```
110         T_upper = h2o_table['T'][idx+1]
111         h_hat_upper = h2o_table['h_hat'][idx+1]
112         h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
113     else:
114         T_lower = h2o_table['T'][idx-1]
115         h_hat_lower = h2o_table['h_hat'][idx-1]
116         h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
117 
118     return h_hat
119 
120 def oh_table_f(Temp):
121     '''
122     :param Temp: temperature in K
123     :return h_hat: enthalpy in J/mol also known as heat of
    formation
124     '''
125     oh_table = pd.read_csv(TABLES / 'OH_table.csv')
126 
127     idx = (oh_table['T']-Temp).abs().idxmin()
128     T_cloest = oh_table['T'][idx]
129     h_hat_close = oh_table['h_hat'][idx]
130     if Temp > T_cloest:
131         T_upper = oh_table['T'][idx+1]
132         h_hat_upper = oh_table['h_hat'][idx+1]
133         h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
134     else:
135         T_lower = oh_table['T'][idx-1]
136         h_hat_lower = oh_table['h_hat'][idx-1]
137         h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
138 
139     return h_hat
140 
141 def h_table_f(Temp):
142     '''
143     :param Temp: temperature in K
144     :return h_hat: enthalpy in J/mol also known as heat of
    formation
145     '''
146     h_table = pd.read_csv(TABLES / 'H_table.csv')
147 
148     idx = (h_table['T']-Temp).abs().idxmin()
```

```
    T_cloest = h_table['T'][idx]
    h_hat_close = h_table['h_hat'][idx]
    if Temp > T_cloest:
        T_upper = h_table['T'][idx+1]
        h_hat_upper = h_table['h_hat'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = h_table['T'][idx-1]
        h_hat_lower = h_table['h_hat'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)

    return h_hat

def h2o_tableT(h_hat):
    '''
    :param h_hat: enthalpy in J/mol also known as heat of formation
    :return Temp: temperature in K
    '''
    h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')

    idx = (h2o_table['h_f']-h_hat).abs().idxmin()
    h_hat_cloest = h2o_table['h_f'][idx]
    T_close = h2o_table['T'][idx]
    if h_hat > h_hat_cloest:
        h_hat_upper = h2o_table['h_f'][idx+1]
        T_upper = h2o_table['T'][idx+1]
        T = T_close + (T_upper-T_close)/(h_hat_upper-h_hat_cloest)*(h_hat-h_hat_cloest)
    else:
        h_hat_lower = h2o_table['h_f'][idx-1]
        T_lower = h2o_table['T'][idx-1]
        T = T_close + (T_lower-T_close)/(h_hat_lower-h_hat_cloest)*(h_hat-h_hat_cloest)

    return T

def h2_tableT(h_hat):
    '''
    :param h_hat: enthalpy in J/mol also known as heat of formation
    :return Temp: temperature in K
    '''
```

```
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')
    h_hat = h_hat/4184
    idx = (h2_table['H*-H*_298']-h_hat).abs().idxmin()
    h_hat_cloest = h2_table['H*-H*_298'][idx]
    T_close = h2_table['T'][idx]
    if h_hat > h_hat_cloest:
        h_hat_upper = h2_table['H*-H*_298'][idx+1]
        T_upper = h2_table['T'][idx+1]
        T = T_close + (T_upper-T_close)/(h_hat_upper-
    h_hat_cloest)*(h_hat-h_hat_cloest)
    else:
        h_hat_lower = h2_table['H*-H*_298'][idx-1]
        T_lower = h2_table['T'][idx-1]
        T = T_close + (T_lower-T_close)/(h_hat_lower-
    h_hat_cloest)*(h_hat-h_hat_cloest)

    return T

def hf_tableT(h_hat):
    '''
    :param h_hat: enthalpy in J/mol also known as heat of
    formation
    :return Temp: temperature in K
    '''
    f2_table = pd.read_csv(TABLES / '
    HydrogenFluoride_HF_table.csv')
    h_hat = h_hat/4184
    idx = (f2_table['H*-H*_298']-h_hat).abs().idxmin()
    h_hat_cloest = f2_table['H*-H*_298'][idx]
    T_close = f2_table['T'][idx]
    if h_hat > h_hat_cloest:
        h_hat_upper = f2_table['H*-H*_298'][idx+1]
        T_upper = f2_table['T'][idx+1]
        T = T_close + (T_upper-T_close)/(h_hat_upper-
    h_hat_cloest)*(h_hat-h_hat_cloest)
    else:
        h_hat_lower = f2_table['H*-H*_298'][idx-1]
        T_lower = f2_table['T'][idx-1]
        T = T_close + (T_lower-T_close)/(h_hat_lower-
    h_hat_cloest)*(h_hat-h_hat_cloest)

    return T


def oh_table_g(Temp):
```

```
    '''
    :param Temp: temperature in K
    :return g_hat: Gibbs free energy in J/mol
    '''
    oh_table = pd.read_csv(TABLES / 'OH_table.csv')

    idx = (oh_table['T']-Temp).abs().idxmin()
    T_closest = oh_table['T'][idx]
    g_hat_close = oh_table['g_hat'][idx]
    if Temp > T_closest:
        T_upper = oh_table['T'][idx+1]
        g_hat_upper = oh_table['g_hat'][idx+1]
        g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = oh_table['T'][idx-1]
        g_hat_lower = oh_table['g_hat'][idx-1]
        g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(T_lower-T_closest)*(Temp-T_closest)

    return g_hat

def h_table_g(Temp):
    '''
    :param Temp: temperature in K
    :return g_hat: Gibbs free energy in J/mol
    '''
    h_table = pd.read_csv(TABLES / 'H_table.csv')

    idx = (h_table['T']-Temp).abs().idxmin()
    T_closest = h_table['T'][idx]
    g_hat_close = h_table['g_hat'][idx]
    if Temp > T_closest:
        T_upper = h_table['T'][idx+1]
        g_hat_upper = h_table['g_hat'][idx+1]
        g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = h_table['T'][idx-1]
        g_hat_lower = h_table['g_hat'][idx-1]
        g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(T_lower-T_closest)*(Temp-T_closest)

    return g_hat

```

```
def h2o_table_g(Temp):
    '''
    :param Temp: temperature in K
    :return g_hat: Gibbs free energy in J/mol
    '''
    h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')

    idx = (h2o_table['T']-Temp).abs().idxmin()
    T_closest = h2o_table['T'][idx]
    g_hat_close = h2o_table['g_hat'][idx]
    if Temp > T_closest:
        T_upper = h2o_table['T'][idx+1]
        g_hat_upper = h2o_table['g_hat'][idx+1]
        g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = h2o_table['T'][idx-1]
        g_hat_lower = h2o_table['g_hat'][idx-1]
        g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(T_lower-T_closest)*(Temp-T_closest)

    return g_hat

def h2_table_g(Temp):
    '''
    :param Temp: temperature in K
    :return g_hat: Gibbs free energy in J/mol
    '''
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')

    idx = (h2_table['T']-Temp).abs().idxmin()
    T_closest = h2_table['T'][idx]
    g_hat_close = h2_table['g_hat'][idx]
    if Temp > T_closest:
        T_upper = h2_table['T'][idx+1]
        g_hat_upper = h2_table['g_hat'][idx+1]
        g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = h2_table['T'][idx-1]
        g_hat_lower = h2_table['g_hat'][idx-1]
        g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(T_lower-T_closest)*(Temp-T_closest)

    return g_hat
```

```
309
310 def h2o_table_cp(Temp):
311     '''
312     :param Temp: temperature in K
313     :return cp_hat: heat capacity in J/mol-K
314     '''
315     h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')
316
317     idx = (h2o_table['T']-Temp).abs().idxmin()
318     T_closest = h2o_table['T'][idx]
319     cp_hat_close = h2o_table['cp_hat'][idx]
320     if Temp > T_closest:
321         T_upper = h2o_table['T'][idx+1]
322         cp_hat_upper = h2o_table['cp_hat'][idx+1]
323         cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
324     else:
325         T_lower = h2o_table['T'][idx-1]
326         cp_hat_lower = h2o_table['cp_hat'][idx-1]
327         cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(
    T_lower-T_closest)*(Temp-T_closest)
328
329     return cp_hat
330
331 def h2_table_cp(Temp):
332     '''
333     :param Temp: temperature in K
334     :return cp_hat: heat capacity in J/mol-K
335     '''
336     h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')
337
338     idx = (h2_table['T']-Temp).abs().idxmin()
339     T_closest = h2_table['T'][idx]
340     cp_hat_close = h2_table['cp_hat'][idx]
341     if Temp > T_closest:
342         T_upper = h2_table['T'][idx+1]
343         cp_hat_upper = h2_table['cp_hat'][idx+1]
344         cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
345     else:
346         T_lower = h2_table['T'][idx-1]
347         cp_hat_lower = h2_table['cp_hat'][idx-1]
348         cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(
    T_lower-T_closest)*(Temp-T_closest)
349
```

```
    return cp_hat

def oh_table_cp(Temp):
    '''
    :param Temp: temperature in K
    :return cp_hat: heat capacity in J/mol-K
    '''
    oh_table = pd.read_csv(TABLES / 'OH_table.csv')

    idx = (oh_table['T']-Temp).abs().idxmin()
    T_cloest = oh_table['T'][idx]
    cp_hat_close = oh_table['cp_hat'][idx]
    if Temp > T_cloest:
        T_upper = oh_table['T'][idx+1]
        cp_hat_upper = oh_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = oh_table['T'][idx-1]
        cp_hat_lower = oh_table['cp_hat'][idx-1]
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)

    return cp_hat

def h_table_cp(Temp):
    '''
    :param Temp: temperature in K
    :return cp_hat: heat capacity in J/mol-K
    '''
    h_table = pd.read_csv(TABLES / 'H_table.csv')

    idx = (h_table['T']-Temp).abs().idxmin()
    T_cloest = h_table['T'][idx]
    cp_hat_close = h_table['cp_hat'][idx]
    if Temp > T_cloest:
        T_upper = h_table['T'][idx+1]
        cp_hat_upper = h_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = h_table['T'][idx-1]
        cp_hat_lower = h_table['cp_hat'][idx-1]
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)
```

```
391
392     return cp_hat
393
394 def MachSolve(gamma, r_A, guess):
395     '''
396     Solve for Mach number given gamma, r_A, and guess
397     :param gamma: the gamma of the gas
398     :param r_A: the ratio of the areas
399     :param guess: the initial guess of the Mach number
400     :return: Mach number
401     '''
402
403     # M = var('M')
404     front_cont = ((gamma+1)/2)**(-(gamma+1)/(2*(gamma-1)))
405     sec_cont = (gamma-1)/2
406     func = lambda M: (((front_cont)*((1+sec_cont*M**2)**((
    gamma+1)/(2*(gamma-1)))))/M) - r_A
407     sol = fsolve(func, guess)
408
409     return sol
410
411 def h2_tableS(Temp):
412     '''
413     :param Temp: temperature in K
414     :return S_hat: entropy in J/mol-K
415     '''
416     h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')
417
418     idx = (h2_table['T']-Temp).abs().idxmin()
419     T_cloest = h2_table['T'][idx]
420     S_hat_close = h2_table['S'][idx]
421     if Temp > T_cloest:
422         T_upper = h2_table['T'][idx+1]
423         S_hat_upper = h2_table['S'][idx+1]
424         S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
425     else:
426         T_lower = h2_table['T'][idx-1]
427         S_hat_lower = h2_table['S'][idx-1]
428         S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
429
430     return S_hat
431
432 def hf_tableS(Temp):
```

```
    '''
    :param Temp: temperature in K
    :return S_hat: entropy in J/mol-K
    '''
    hf_table = pd.read_csv(TABLES / '
    HydrogenFluoride_HF_table.csv')

    idx = (hf_table['T']-Temp).abs().idxmin()
    T_closest = hf_table['T'][idx]
    S_hat_close = hf_table['S'][idx]
    if Temp > T_closest:
        T_upper = hf_table['T'][idx+1]
        S_hat_upper = hf_table['S'][idx+1]
        S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = hf_table['T'][idx-1]
        S_hat_lower = hf_table['S'][idx-1]
        S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(
    T_lower-T_closest)*(Temp-T_closest)

    return S_hat

def f2_tableS(Temp):
    '''
    :param Temp: temperature in K
    :return S_hat: entropy in J/mol-K
    '''
    f2_table = pd.read_csv(TABLES / 'Fluorine_F2_table.csv')

    idx = (f2_table['T']-Temp).abs().idxmin()
    T_closest = f2_table['T'][idx]
    S_hat_close = f2_table['S'][idx]
    if Temp > T_closest:
        T_upper = f2_table['T'][idx+1]
        S_hat_upper = f2_table['S'][idx+1]
        S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = f2_table['T'][idx-1]
        S_hat_lower = f2_table['S'][idx-1]
        S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(
    T_lower-T_closest)*(Temp-T_closest)

    return S_hat
```

```
473 # Me = MachSolve(1.237115, 77.5426, 5)
474 # print('Me = ', Me)
```